

☀️ Class Test 1 - Computer Graphics and Geometric Modeling (2026) ☀️

💎 Complete Detailed Solutions for Set-2 💎

📌 QUESTION 1

🌸 Part (a)

Given the center point coordinate (15, 10) and a radius of 20 units, generate all the points (in the 3rd octant) by using both the Bresenham and Midpoint circle drawing algorithms. Also, compare the efficiency of both algorithms in terms of the number of calculations and the computational complexity involved in plotting the points. [Marks: 8]

💡 Mathematical Formulations and Steps

Before applying the final translation to the screen center point $(X_c, Y_c) = (15, 10)$, we first calculate the baseline pixel positions centered at the origin $(0, 0)$ for the standard baseline octant (where x ranges from 0 up to y , meaning $x \leq y$).

To transform these baseline coordinates into the true **3rd octant** (where x is negative, y is positive, and $|x| \leq y$), the standard octant symmetry mapping is:

- Origin Point mapped to 3rd Octant: $(-x, y)$

Finally, we apply translation to shift the points to the given center:

- $X_{\text{screen}} = -x + X_c = -x + 15$
- $Y_{\text{screen}} = y + Y_c = y + 10$

1 Midpoint Circle Drawing Algorithm

- **Initial Values:** $x = 0, y = 20$
- **Initial Decision Parameter:** $p_0 = 1 - R = 1 - 20 = -19$
- **Parameter Update Rules:**
 - If $p_k < 0$: $p(k+1) = p_k + 2x + 3$. Next point is $(x + 1, y)$.
 - If $p_k \geq 0$: $p(k+1) = p_k + 2x - 2y + 5$. Next point is $(x + 1, y - 1)$.

📊 Tracing Table (Midpoint Method)

Step k	Decision Parameter p_k	Point at (0,0)	3rd Octant Mapped $(-x, y)$	Final Screen Coordinate $(-x+15, y+10)$
0	Initial	(0, 20)	(0, 20)	(15, 30)

Step k	Decision Parameter pk	Point at (0,0)	3rd Octant Mapped (-x, y)	Final Screen Coordinate (-x+15, y+10)
1	-19	(1, 20)	(-1, 20)	(14, 30)
2	-16	(2, 20)	(-2, 20)	(13, 30)
3	-11	(3, 20)	(-3, 20)	(12, 30)
4	-4	(4, 20)	(-4, 20)	(11, 30)
5	5	(5, 19)	(-5, 19)	(10, 29)
6	-22	(6, 19)	(-6, 19)	(9, 29)
7	-9	(7, 19)	(-7, 19)	(8, 29)
8	6	(8, 18)	(-8, 18)	(7, 28)
9	-13	(9, 18)	(-9, 18)	(6, 28)
10	6	(10, 17)	(-10, 17)	(5, 27)
11	-7	(11, 17)	(-11, 17)	(4, 27)
12	16	(12, 16)	(-12, 16)	(3, 26)
13	9	(13, 15)	(-13, 15)	(2, 25)
14	6	(14, 14)	(-14, 14)	(1, 24)

2 Bresenham's Circle Drawing Algorithm

- **Initial Values:** $x = 0, y = 20$
- **Initial Decision Parameter:** $d_0 = 3 - 2R = 3 - 40 = -37$
- **Parameter Update Rules:**
 - If $d_k < 0$: $d(k+1) = d_k + 4x + 6$. Next point is $(x + 1, y)$.
 - If $d_k \geq 0$: $d(k+1) = d_k + 4(x - y) + 10$. Next point is $(x + 1, y - 1)$.

Tracing Table (Bresenham Method)

Step k	Decision Parameter dk	Point at (0,0)	3rd Octant Mapped (-x, y)	Final Screen Coordinate (-x+15, y+10)
0	Initial	(0, 20)	(0, 20)	(15, 30)
1	-37	(1, 20)	(-1, 20)	(14, 30)
2	-31	(2, 20)	(-2, 20)	(13, 30)
3	-21	(3, 20)	(-3, 20)	(12, 30)
4	-7	(4, 20)	(-4, 20)	(11, 30)
5	11	(5, 19)	(-5, 19)	(10, 29)

Step k	Decision Parameter dk	Point at (0,0)	3rd Octant Mapped (-x, y)	Final Screen Coordinate (-x+15, y+10)
6	-43	(6, 19)	(-6, 19)	(9, 29)
7	-17	(7, 19)	(-7, 19)	(8, 29)
8	13	(8, 18)	(-8, 18)	(7, 28)
9	-25	(9, 18)	(-9, 18)	(6, 28)
10	13	(10, 17)	(-10, 17)	(5, 27)
11	-13	(11, 17)	(-11, 17)	(4, 27)
12	33	(12, 16)	(-12, 16)	(3, 26)
13	19	(13, 15)	(-13, 15)	(2, 25)
14	13	(14, 14)	(-14, 14)	(1, 24)

Efficiency and Complexity Analysis

1. **Computational Complexity:** Both algorithms require $O(N)$ time complexity, where N is the number of pixels plotted along the circle arc segment. They completely replace expensive floating-point or trigonometric calculations with fast integer additions, subtractions, and bit shifts.
2. **Computational Counts:** The Midpoint method increments the decision parameter using linear variables ($2x + 3$ or $2x - 2y + 5$), while Bresenham's uses structural steps ($4x + 6$ or $4(x - y) + 10$). They make identical pixel-plotting choices because Bresenham's parameter is mathematically related to the Midpoint parameter by $d = 2p - 1$. Therefore, operational speed is identical on modern systems.

Part (b)

How long would it take to load a 640x480 frame buffer with 12 bits per pixel, if 10^5 bits can be transferred per second? How long would it take to load a 24-bit per pixel frame buffer with a resolution of 1280x1024 using this same transfer rate? [Marks: 3]

 **Solution:**

Scenario 1: 640x480 Monitor

1. Total pixels = $640 * 480 = 307,200$ pixels
2. Total memory bits = $307,200 \text{ pixels} * 12 \text{ bits/pixel} = 3,686,400$ bits
3. Transfer speed = 10^5 bits per second = 100,000 bits/sec
4. Time taken = $3,686,400 / 100,000 = \mathbf{36.864 \text{ seconds}}$

Scenario 2: 1280x1024 Monitor

1. Total pixels = $1280 * 1024 = 1,310,720$ pixels
2. Total memory bits = $1,310,720 \text{ pixels} * 24 \text{ bits/pixel} = 31,457,280$ bits
3. Transfer speed = 10^5 bits per second = 100,000 bits/sec

4. Time taken = $31,457,280 / 100,000 = 314.573$ seconds

QUESTION 2

Part (a)

Given a cubic Bezier curve defined by the control points $P_1(-8,0,0)$, $P_2(-4,5,0)$, $P_3(4,-5,0)$ and $P_4(8,0,0)$. Determine the geometric coefficient matrix that reproduces this curve in the cubic Hermite form. [Marks: 7]

Solution:

A cubic Hermite curve is defined by its two endpoints and the tangent vectors at those two endpoints. The geometric coefficient matrix GH is structured as: $GH = [P(0), P(1), P'(0), P'(1)]$ transposed

For a cubic Bezier curve with control points P_1, P_2, P_3, P_4 :

1. The start point $P(0)$ corresponds exactly to the first control point:
 - $P(0) = P_1 = (-8, 0, 0)$
2. The end point $P(1)$ corresponds exactly to the last control point:
 - $P(1) = P_4 = (8, 0, 0)$
3. The starting tangent vector $P'(0)$ is given by:
 - $P'(0) = 3 * (P_2 - P_1) = 3 * (-4 - (-8), 5 - 0, 0 - 0) = 3 * (4, 5, 0) = (12, 15, 0)$
4. The ending tangent vector $P'(1)$ is given by:
 - $P'(1) = 3 * (P_4 - P_3) = 3 * (8 - 4, 0 - (-5), 0 - 0) = 3 * (4, 5, 0) = (12, 15, 0)$

Final Hermite Geometric Matrix (GH):

```
[ -8,  0,  0 ] <- Start Position P(0)
[  8,  0,  0 ] <- End Position P(1)
[ 12, 15,  0 ] <- Start Tangent Vector P'(0)
[ 12, 15,  0 ] <- End Tangent Vector P'(1)
```

Part (b)

Obtain no. of segments, no. of control points in each segment, total control points, degree and order of the B-spline curve defined by $n=5$ and $k=3$. [Marks: 3]

Solution:

Given parameters: $n = 5$ and $k = 3$.

1. **Total Number of Control Points:** Given by $(n + 1) = 5 + 1 = 6$ control points.
2. **Order of the Curve (k):** Explicitly given as 3.
3. **Degree of the Polynomial (p):** $p = k - 1 = 3 - 1 = 2$ (Quadratic spline).
4. **Number of Control Points in Each Segment:** Equal to the order $k = 3$ control points.

5. **Number of Segments:** Formula is $(n + 1) - k + 1 = 6 - 3 + 1 = 4$ segments.



QUESTION 3

The lower left-hand and upper right-hand corners of a rectangular clipping window are at $(-3, 1)$ and $(2, 6)$, respectively. Using Sutherland-Cohen algorithm, check and find the visibility portion of the quadrilateral EFGH for $E(-4,2)$, $F(-1,7)$, $G(-1,5)$ and $H(-2,3)$ in the clipping window. [Marks: 9]

Solution:

1 Region Boundary Codes

Outcodes are assigned using 4 bits: **[Top, Bottom, Right, Left]**. Window Limits: $X_{min} = -3$, $X_{max} = 2$, $Y_{min} = 1$, $Y_{max} = 6$

- **Point E(-4, 2):** $X < X_{min} \rightarrow$ Outcode is **0001** (Left)
- **Point F(-1, 7):** $Y > Y_{max} \rightarrow$ Outcode is **1000** (Top)
- **Point G(-1, 5):** Inside all boundaries $\rightarrow$ Outcode is **0000** (Inside)
- **Point H(-2, 3):** Inside all boundaries $\rightarrow$ Outcode is **0000** (Inside)

2 Segment-by-Segment Clipping Execution

● Segment EF (E(-4, 2) to F(-1, 7))

- Codes: $E = 0001$, $F = 1000$. Logical AND = 0000 (Partially visible candidate).
- Clip E against Left boundary ($X = -3$):
 - Slope $m = (7 - 2) / (-1 - (-4)) = 5 / 3 = 1.667$
 - $Y_{intercept} = 2 + 1.667 * (-3 - (-4)) = 3.667$
 - New point $E' = (-3, 3.667)$ [Code 0000]
- Clip F against Top boundary ($Y = 6$):
 - $X_{intercept} = -1 + (6 - 7) / 1.667 = -1.6$
 - New point $F' = (-1.6, 6)$ [Code 0000]
- **Visible Line Component: E'(-3, 3.667) to F'(-1.6, 6).**

● Segment FG (F(-1, 7) to G(-1, 5))

- Codes: $F = 1000$, $G = 0000$. Logical AND = 0000.
- Clip F against Top boundary ($Y = 6$):
 - X is constant at -1.
 - New point $F'' = (-1, 6)$ [Code 0000]
- **Visible Line Component: F''(-1, 6) to G(-1, 5).**

● Segment GH (G(-1, 5) to H(-2, 3))

- Codes: $G = 0000$, $H = 0000$.
- Both endpoints are 0000 $\rightarrow$ Trivial Accept.

- **Visible Line Component: G(-1, 5) to H(-2, 3).**

● **Segment HE (H(-2, 3) to E(-4, 2))**

- Codes: H = 0000, E = 0001. Logical AND = 0000.
 - Clip E against Left boundary (X = -3):
 - Slope $m = (2 - 3) / (-4 - (-2)) = -1 / -2 = 0.5$
 - $Y_{\text{intersect}} = 3 + 0.5 * (-3 - (-2)) = 2.5$
 - New point E'' = (-3, 2.5) [Code 0000]
 - **Visible Line Component: H(-2, 3) to E''(-3, 2.5).**
-

🎯 Final Summary of Visible Quadrilateral

The clipped visible portion forms a beautiful polygon with the following sequential vertices:

1. **E'(-3, 3.667)**
2. **F'(-1.6, 6)**
3. **F''(-1, 6)**
4. **G(-1, 5)**
5. **H(-2, 3)**
6. **E''(-3, 2.5)**